Here's the scaffold I recommended for Project Reachy_Local once we refactored it to host both the backend (FastAPI + Postgres + TAO) and the frontend (Streamlit web app) inside a single Windsurf-friendly monorepo.

It's designed for development inside Windsurf/VS Code with Continue + Claude Code + Cascade integration and Docker/dev-containers support.

🗂 Reachy_Local 08.4.x Scaffold

```
reachy_local/
├── apps/
│   ├── api/                # FastAPI backend ("Media Mover")
│   │   ├── app/
│   │   │   ├── main.py         # FastAPI entrypoint (root_path=/api/media)
│   │   │   ├── models.py       # SQLAlchemy ORM models
│   │   │   ├── schemas.py      # Pydantic v2 models
│   │   │   ├── db.py           # DB session + engine config
│   │   │   ├── routes/
│   │   │   │   ├── ingest.py
│   │   │   │   ├── label.py
│   │   │   │   ├── promote.py
│   │   │   │   ├── stats.py
│   │   │   │   └── healthz.py
│   │   │   ├── utils/
│   │   │   │   ├── thumbnails.py
│   │   │   │   ├── files.py
│   │   │   │   └── validators.py
│   │   │   └── __init__.py
│   │   ├── tests/
│   │   │   └── test_endpoints.py
│   │   ├── requirements.txt
│   │   ├── alembic/            # migrations
│   │   ├── Dockerfile
│   │   └── .env.example
│   │
│   └── web/                # Streamlit web UI
```

```
│     ├── main_app.py          # entry point (emotion labeling dashboard)
│     ├── components/          # Streamlit custom widgets
│     │   ├── video_player.py
│     │   └── stats_panel.py
│     ├── pages/
│     │   ├── 01_Generate.py
│     │   ├── 02_Label.py
│     │   ├── 03_Train.py
│     │   └── 04_Deploy.py
│     ├── api_client.py        # REST client for FastAPI endpoints
│     ├── requirements.txt
│     └── .streamlit/config.toml
│
├── tao/                       # NVIDIA TAO & DeepStream assets
│   ├── specs/
│   │   ├── emotion_train_2cls.yaml
│   │   └── emotion_eval.yaml
│   ├── notebooks/
│   │   └── tao_finetune.ipynb
│   └── Docker/
│     ├── tao4_train.sh
│     ├── tao5_export.sh
│     └── tao_mounts.json
│
├── data/
│   ├── manifests/
│   │   ├── train_manifest.json
│   │   └── test_manifest.json
│   └── videos/
│     ├── temp/
│     ├── train/
│     ├── test/
│     └── thumbs/
│
├── infra/
│   ├── nginx/
```

```
│   │   ├── media_mover.conf
│   │   └── ssl/
│   ├── systemd/
│   │   └── fastapi-media.service
│   ├── scripts/
│   │   ├── bootstrap_reachy_db_and_media.sh
│   │   ├── deploy.sh
│   │   ├── smoke.sh
│   │   └── backup.sh
│   ├── docker-compose.yml
│   └── devcontainer.json
│
├── docs/
│   ├── requirements_08.4.2.md
│   ├── AGENTS_08.4.2.md
│   ├── MODEL_SPEC.md
│   ├── ARCHITECTURE.png
│   └── WORKFLOW.md
│
├── tests/
│   ├── unit/
│   ├── integration/
│   └── regression/
│
├── .vscode/                # or `.windsurf/`
│   ├── settings.json
│   ├── launch.json
│   └── tasks.json
│
├── .gitignore
├── README.md
├── LICENSE
└── pyproject.toml          # unified lint/test/build config
```

💡 **Design Notes**

- **Two apps, one monorepo: apps/api and apps/web share the same virtual env via devcontainer.json.**
- **Database first: Postgres stores metadata + URLs; filesystem holds actual videos; Nginx exposes /videos & /thumbs.**
- **Agentic loop: n8n orchestrates ingestion → labeling → promotion → training → evaluation → deployment.**
- **CI/CD: GitHub Actions run lint + pytest + docker build; manual approval for deployment.**
- **Reproducibility: All training/spec versions pinned in tao/specs/ and referenced by MLflow runs.**

This scaffold gave you clean boundaries:

- **apps/ → developer-facing services**
- **tao/ → model training/export**
- **infra/ → deployment and system integration**
- **docs/ → auditable configuration & policy**